

PROGRAMMAZIONE II

LIBRERIE E STRINGHE JAVA

Michele Pasqua, PhD



UNIVERSITÀ
di **VERONA**
Dipartimento
di **INFORMATICA**

A.A. 2025/2026

LIBRERIE STANDARD

Java fornisce un gran numero di librerie standard (o pacchetti)

- ▶ `java.lang`
- ▶ `java.io`
- ▶ `java.math`
- ▶ `java.awt`
- ▶ `java.util`
- ▶ ...

// funzionalità runtime di base

// supporto di input/output

// operazioni matematiche

// supporto GUI di base

// funzioni di utilità

Per esempio, `java.lang.String` fornisce il supporto per le stringhe

IMPORTAZIONE DI LIBRERIE

Prima dell'uso, le librerie o le classi devono essere importate

```
import java.util.*;    0    import java.util.Scanner;
```

Si noti che

- ▶ La libreria `java.lang` è sempre importata di default
- ▶ Le librerie e le classi devono essere disponibili nel classpath

IMPORTAZIONE DI LIBRERIE STANDARD

```
import java.util.Scanner; // importa una libreria standard      1
                                                                    2
public class MainScanner {                                       3
                                                                    4
    public static void main(String[] args) {                    5
        Scanner myScanner = new Scanner(System.in);            6
                                                                    7
        System.out.println("Enter username: ");                8
        String userName = myScanner.nextLine();                 9
                                                                    10
        System.out.println("The username you typed is: " + userName); 11
        myScanner.close();                                       12
    }                                                            13
                                                                    14
}                                                                15
```

E questo?



Compilare con `javac -cp . MainScanner.java`

IMPORTAZIONE DI CLASSI DEFINITE DALL'UTENTE

Per ora, inserire tutti i file .class nello stesso classpath

- Vedremo più avanti come creare pacchetti e importare librerie definite dall'utente

```
// non servono direttive import se le classi sono nello stesso classpath      1
                                                                              2
public class MainImport {                                                    3
                                                                              4
    public static void main(String[] args) {                                  5
        System.out.println("We will import MainScanner and run its main");    6
        MainScanner.main( // il main è un metodo statico                      7
    }                                                                           8
                                                                              9
}                                                                              10
```

Compilare con `javac -cp . MainImport.java`

ALCUNE CLASSI STANDARD DI JAVA

La classe `java.lang.System` fornisce le costanti

- ▶ `err`, che fa riferimento allo standard error
- ▶ `in`, che fa riferimento allo standard input
- ▶ `out`, che fa riferimento allo standard output

La classe `java.lang.System` fornisce il metodo statico `currentTimeMillis()`

- ▶ Restituisce un `long`
- ▶ Restituisce i millisecondi trascorsi dal 1° gennaio 1970 (UTC) a mezzanotte

TEMPO UNIX



ALCUNE CLASSI STANDARD DI JAVA (CONT.)

Per usare la classe `java.util.Scanner`

- ▶ `sc = new Scanner(source)`, un oggetto scanner deve essere collegato a una sorgente
- ▶ `sc.close()`, un oggetto scanner dovrebbe essere chiuso dopo l'uso

La classe `java.util.Scanner` fornisce i metodi

- ▶ `nextLine()`, legge una riga dalla sorgente come `String`
- ▶ `nextInt()`, legge un intero dalla sorgente come `int`
- ▶ `nextFloat()`, legge un decimale dalla sorgente come `float`
- ▶ ... verrà lanciata un'eccezione in caso di tipo non corrispondente

STRINGHE IN JAVA



CLASSE STRING

In Java, le stringhe sono istanze della classe `String`

Possiamo creare stringhe di due tipi

- ▶ Stringhe vuote, con `String str = new String();`
- ▶ Stringhe non vuote, con `String str = new String("una_stringa");`

Qui `"una_stringa"` è una stringa costante

- ▶ Il compilatore la trasforma implicitamente in un oggetto `String`
- ▶ Si può anche scrivere `String str = ""`; e `String str = "una_stringa"`;

CLASSE STRING (CONT.)

In Java, le stringhe non sono array di caratteri (terminati da null)

```
char carr[] = {'h', 'e', 'l', 'l', 'o'};  ≠  String str = "hello";
```

Tutte le stringhe sono oggetti `String`

- ▶ Possiamo avere `String str = null;`
- ▶ Non possiamo modificare le stringhe, gli oggetti `String` sono immutabili
- ▶ Ma possiamo copiare le stringhe

```
String s = new String("abc");           1
String t = new String(s);                2
// s e t fanno riferimento a due oggetti diversi, che rappresentano la stessa stringa 3
```

CONCATENAZIONE DI STRINGHE

L'unico operatore disponibile sulle stringhe è +, che esegue la concatenazione

```
String s = new String("Hello_");           1
String t = s + "World";                     2
System.out.println(t + "!")                 3
// l'output sarà 'Hello World!'             4
```

Tutte le altre operazioni sulle stringhe sono implementate come metodi della classe `String`

OPERAZIONI SULLE STRINGHE

Il numero di elementi in una stringa viene calcolato con

`str.length()`

```
String str = "";
System.out.println(str.length()); // l'output sarà '0'

String str = "hello";
System.out.println(str.length()); // l'output sarà '5'

System.out.println("abc".length()); // l'output sarà '3'
```

1
2
3
4
5
6
7

OPERAZIONI SULLE STRINGHE (CONT.)

È possibile accedere all'i-esimo elemento di una stringa con

```
str.charAt(i)
```

L'indicizzazione parte da 0, se l'indice è fuori dai limiti viene lanciata un'eccezione

```
String str = "hello";                                     1
System.out.println(str.charAt(0)); // l'output sarà 'h'    2
System.out.println(str.charAt(4)); // l'output sarà 'o'    3
System.out.println(str.charAt(5)); // la JVM lancerà un'eccezione 4
System.out.println(str.charAt(5)); // la JVM lancerà un'eccezione 5
System.out.println(str.charAt(5)); // la JVM lancerà un'eccezione 6
System.out.println(str.charAt(5)); // la JVM lancerà un'eccezione 7
```

OPERAZIONI SULLE STRINGHE (CONT.)

È possibile accedere all'i-esimo elemento di una stringa con

```
str.charAt(i)
```

Gli oggetti String sono immutabili, non è possibile modificare gli elementi di una stringa

```
String str = "hello";
```

1

```
System.out.println(str.charAt(3)); // l'output sarà 'l'
```

2

3

```
str.charAt(3) = 'm'; // errore di compilazione
```

4

5

CONFRONTO TRA STRINGHE

Essendo oggetti, l'operatore `==` viene applicato ai riferimenti quando si confrontano le stringhe

`s == t` è vero solo se `s` e `t` puntano allo stesso oggetto

```
String s = "hello";           1
String t = new String("hello"); 2
String r = s;                  3

System.out.println(s == t);    4 // l'output sarà 'false' 5
                                6
System.out.println(s == r);    7 // l'output sarà 'true'
```

CONFRONTO TRA STRINGHE (CONT.)

Per confrontare le stringhe, usare il metodo `equals` fornito dalla classe `String`

`s.equals(t)` è vero quando `s` e `t` rappresentano la stessa stringa

```
String s = "hello";           1
String t = new String("hello"); 2
String r = s;                  3

System.out.println(s.equals(t)); // l'output sarà 'true' 4
                                   5
System.out.println(s.equals(r)); // l'output sarà 'true' 6
                                   7
```

CONFRONTO TRA STRINGHE (CONT.)

Prestare attenzione alle ottimizzazioni del compilatore

! Usare sempre `equals` per confrontare le stringhe

```
String s = "hello";           1
String t = "hello";           2
String r = s;                  3

System.out.println(s == r);    4 // l'output sarà 'true', previsto 5
System.out.println(s == t);    6 // l'output sarà 'true', perché?   7
```

ALTRE OPERAZIONI SULLE STRINGHE

L'indice (prima occorrenza) di un carattere *c* in una stringa viene calcolato con

`str.indexOf(c)`

Il metodo restituisce -1 quando il carattere non è presente

```
String str = "hello";                                     1
                                                                    2
System.out.println(str.indexOf('e')); // l'output sarà '1'  3
                                                                    4
System.out.println(str.indexOf('j')); // l'output sarà '-1'  5
```

ALTRE OPERAZIONI SULLE STRINGHE (CONT.)

La sottostringa tra i limiti `start` ed `end` in una stringa viene calcolata con

```
str.substring(start, end)
```

Il metodo restituisce la stringa che inizia all'indice `start` e termina all'indice `end-1`

```
String str = "hello";  
  
System.out.println(str.substring(1, 3)); // l'output sarà 'el'
```

1
2
3
4
5
6
7

ALTRE OPERAZIONI SULLE STRINGHE (CONT.)

La sottostringa tra i limiti `start` ed `end` in una stringa viene calcolata con

```
str.substring(start, end)
```

Se `end` viene omesso, la sottostringa prosegue fino alla fine della stringa

```
String str = "hello";  
  
System.out.println(str.substring(1, 3)); // l'output sarà 'el'  
  
System.out.println(str.indexOf(2)); // l'output sarà 'llo'
```

ALTRE OPERAZIONI SULLE STRINGHE (CONT.)

La sottostringa tra i limiti `start` ed `end` in una stringa viene calcolata con

```
str.substring(start, end)
```

Se `start` o `end` sono fuori dai limiti viene lanciata un'eccezione

```
String str = "hello";                                1
                                                         2
System.out.println(str.substring(1, 3)); // l'output sarà 'el' 3
                                                         4
System.out.println(str.indexOf(2)); // l'output sarà 'llo'    5
                                                         6
System.out.println(str.indexOf(2, 6)); // la JVM lancerà un'eccezione 7
```

ALTRE OPERAZIONI SULLE STRINGHE (CONT.)

Sostituisce la stringa `target` con la stringa `replacement` in una stringa `str` con

```
str.replace(target, replacement)
```

La sostituzione viene applicata da sinistra a destra (eventualmente più volte)

```
String str = "hello";           1
                                  2
System.out.println(str.replace("lo", "ix")); // produce 'helix' 3
                                  4
System.out.println(str.replace("o", ""));    // produce 'hell' 5
                                  6
System.out.println(str.replace("l", "ll"));  // produce 'helllllo' 7
```


CAST IMPLICITO

In Java, è possibile concatenare stringhe e interi (o altri tipi primitivi)

- ▶ Il compilatore esegue un cast implicito da `int` a `String`
- ▶ Quando si usa `+`, il compilatore gestisce automaticamente i valori `null`

```
String str = "Result_=" + 3;           1
System.out.println(str); // l'output sarà 'Result = 3' 2
System.out.println(str + null); // l'output sarà 'Result = null' 3
                                     4
                                     5
```

In realtà, il compilatore chiama il metodo `valueOf()` della classe `String`

STRINGHE ALLA C IN JAVA

La classe `String` fornisce il metodo `format()`

- ▶ La sintassi è simile alle stringhe di formato del C // usato in printf e simili

```
String str = "Result";
int num = 3;

String formatStr = String.format("%s = %d", str, num);

System.out.println(formatStr); // l'output sarà 'Result = 3'
```

STRINGHE MUTABILI

Il pacchetto `java.lang` fornisce la classe `StringBuilder`

- Crea oggetti che contengono una sequenza mutabile di caratteri

String immutabile

```
String str = "abc"; 1  
str = str + "def"; // nessun append effettivo, viene creato un nuovo oggetto 2
```

StringBuilder mutabile

```
StringBuilder sb = new StringBuilder("abc"); 1  
sb.append("def"); // append sullo stesso oggetto 2
```

STRINGHE MUTABILI (CONT.)

Utile per costruire dinamicamente stringhe

```
String printArray(int[] arr)    // se arr è '[1, 5, 6]'           1
    StringBuilder sb = new StringBuilder();                       2
                                                                    3
    sb.append("Array(").append(arr.length).append(") [");       4
    for (int i = 0; i < arr.length; i++) {                       5
        sb.append(" ").append(arr[i]);                           6
    }                                                             7
    sb.append(" ]");                                              8
                                                                    9
    return sb.toString();    // il metodo restituirà 'Array(3)[ 1 5 6 ]' 10
}
```

STRINGHE MUTABILI (CONT.)

Alcune operazioni aggiuntive

// tutte in fluent notation

Eliminare una sottostringa entro i limiti (escluso a destra) con

```
StringBuilder delete(int start, int end);
```

Inserire una stringa alla posizione con

```
StringBuilder insert(int index, String str);
```

Invertire una stringa con `StringBuilder reverse()`;



DOMANDE E RISPOSTE

